

Memory Systems Architecture – Assignment #1 Report

Name: Jiahao Chen

Machine: Apple MacBook Air (Apple M4)

1. Overview

This assignment experimentally characterizes the memory hierarchy of a modern Apple M4 system. The goal is to determine cache hierarchy levels, cache capacities, cache line size, approximate main memory latency, and the effective associativity of the system-level cache (SLC).

2. Experimental Methodology

All experiments use a pointer-chasing microbenchmark, where each memory access depends on the previous one. This prevents hardware prefetching and out-of-order execution from hiding true memory latency. Randomized access patterns are used to eliminate spatial locality.

Timing was performed using `clock_gettime(CLOCK_MONOTONIC)`. The program was compiled with `-O2`, `-fno-tree-vectorize`, and `-march=native` to prevent aggressive compiler optimizations that could distort results.

3. Cache Hierarchy and Capacity

By increasing the working set size from small memory regions to tens of megabytes, distinct latency plateaus and jumps were observed, corresponding to different cache levels and main memory.

The lowest latency region corresponds to L1 cache hits. A noticeable latency increase indicates transitions to the L2 cache, followed by a higher-latency region corresponding to the shared system-level cache (SLC). A large jump in latency marks the transition to unified DRAM.

--- PART 1: Cache Capacity & Levels ---	
Size(KB), Latency(ns)	
1, 1.45	192, 2.22
1, 1.30	256, 2.93
2, 1.09	384, 3.71
3, 0.95	512, 4.76
4, 0.88	768, 5.05
6, 0.82	1024, 4.83
8, 0.78	1536, 5.32
12, 0.75	2048, 4.76
16, 0.73	3072, 5.22
24, 0.70	4096, 6.07
32, 0.68	6144, 6.95
48, 0.68	8192, 7.86
64, 0.68	12288, 7.35
96, 0.68	16384, 12.13
128, 0.68	24576, 33.65
	32768, 48.78
	49152, 60.20
	65536, 75.06

4. Cache Line Size

Cache line size was determined using a strided access pattern over a fixed-size working set. Latency remains low for small strides and increases sharply once the stride exceeds the cache line size.

A clear inflection point occurs at a stride of 64 bytes, indicating that the cache line size on the Apple M4 system is 64 bytes.

```

--- PART 2: Cache Line Size ---
Stride(Bytes), Latency(ns)
8, 0.70
16, 0.69
32, 0.93
64, 1.33
128, 1.55
256, 1.77
512, 4.86

```

5. System-Level Cache Associativity

To estimate system-level cache associativity, a pointer chain was constructed across multiple memory blocks spaced far apart in memory but mapped to the same cache set. As the number of blocks increases, conflict misses occur once the associativity limit is exceeded, resulting in higher access latency.

The results indicate that the effective associativity of the system-level cache is approximately between 8 and 12 ways. The transition is gradual due to cache banking, address hashing, and non-LRU replacement policies used in Apple Silicon.

```

--- PART 3: LLC Associativity ---
Ways, Latency(ns)
1, 0.25
2, 0.68
3, 1.91
4, 2.52
5, 2.15
6, 3.28
7, 3.62
8, 3.89
9, 4.92
10, 5.64
11, 6.23
12, 6.67
13, 6.55
14, 5.99
15, 6.07
16, 6.14

```

```

17, 6.23
18, 6.22
19, 6.68
20, 6.60
21, 6.44
22, 6.97
23, 8.28
24, 7.49
25, 7.47
26, 6.42
27, 6.64
28, 6.67
29, 6.45
30, 7.78
31, 7.94
32, 8.04

```

6. Apple Silicon Considerations

The Apple M4 is an ARM-based system-on-chip with a unified memory architecture and a shared system-level cache. Unlike traditional x86 processors, the last-level cache is banked and heavily hashed, which causes cache boundaries to appear as gradual transitions rather than sharp cliffs. All reported values therefore represent effective cache behavior.

7. Conclusion

This assignment demonstrates that carefully designed pointer-chasing microbenchmarks can successfully reveal cache hierarchy properties on modern Apple Silicon systems. The results provide realistic estimates of cache capacities, cache line size, memory latency, and effective system-level cache associativity.